



Sorting

- One fundamental problem of computer science is ordering a list of items.
- Many solutions exist to this problem, known as *sorting algorithms*.
- Some sorting algorithms are simple and intuitive, such as the *bubble sort*. Others, such as the *quick sort* are extremely complicated, but produce fast results.

Sorting Unit III. 1 Dr. Rabins Porwal

1



Sorting

Algorithms are divided into two categories:

- Internal Sorts

and

- External sorts

Sorting Unit III. 2 Dr. Rabins Porwal

2



Sorting

Internal Sort:

- Any sort algorithm which uses *main memory exclusively* during the sort.
- This assumes high-speed random access to all memory.

Sorting Unit III. 3 Dr. Rabins Porwal

3



Sorting

External Sort:

- Any sort algorithm which uses *external memory*, such as tape or disk, during the sort.

Sorting Unit III. 4 Dr. Rabins Porwal

4



Sorting

Note:

*Algorithms may read the initial values from magnetic tape or write sorted values to disk, but **this is not using external memory** during the sort. Note that even though virtual memory may mask the use of disk, **sorting sets of data much larger than main memory may be much faster using an explicit external sort.***

Sorting Unit III. 5 Dr. Rabins Porwal

5



Sorting

Sort Stable

A sort algorithm is said to be "**stable**" if multiple items which compare as equal will stay in **the same order** they were in after a sort.

Sorting Unit III. 6 Dr. Rabins Porwal

6

Unit III. 7

Sorting

Dr. Rabins Porwal

Sorting

Example:

Tom Greene
Ann George
Peter George
Arthur George
Joe Summers

Sorting on surnames →

Ann George
Peter George
Arthur George
Tom Greene
Joe Summers

All the George names maintain their position, although Arthur is less than Peter

7

Unit III. 8

Sorting

Dr. Rabins Porwal

Internal Sorting

Bucket Sort

- Possibly the *simplest distribution sorting* algorithm.
- The essential requirement is that the size of the universe, from which the elements to be sorted are drawn, is a small, fixed constant, say m .

Cont'd...

8

Unit III. 9

Sorting

Dr. Rabins Porwal

Internal Sorting

Bucket Sort

For example, suppose that we are sorting elements drawn from $\{0, 1, \dots, m-1\}$, i.e., the set of integers in the interval $[0, m-1]$.

Bucket sort uses m counters. The i^{th} counter keeps track of the *number of occurrences* of the i^{th} element of the universe.

9

Unit III. 10

Sorting

Dr. Rabins Porwal

Internal Sorting

Bucket Sort (Example)

3	1	4	1	5	0	2	6	5	4
data									
0	2	1	1	2	2	1	0	0	1
counts									
0	1	2	3	4	5	6	7	8	9
1	1	2	3	4	4	5	5	6	0
data									

10

Unit III. 11

Sorting

Dr. Rabins Porwal

Internal Sorting

Bubble Sort

The oldest and simplest sort in use.

Unfortunately, also the slowest.

Works by comparing each item in the list with the item next to it, and swapping them if required.

11

Unit III. 12

Sorting

Dr. Rabins Porwal

Internal Sorting

Bubble Sort

The algorithm repeats this process until it makes a pass all the way through the list without swapping any items.

This causes larger values to "bubble" to the end of the list while smaller values "sink" towards the beginning of the list.

12



Internal Sorting

Insertion Sorting

- Works just like its name suggests - it inserts each item into its proper place in the final list.
- The simplest implementation of this requires two list structures - the source list and the list into which sorted items are inserted.

Sorting

Unit III. 19

Dr. Rabins Porwal

19



Internal Sorting

Insertion Sorting

- To save memory, most implementations use an in-place sort that works by moving the current item past the already sorted items and repeatedly swapping it with the preceding item until it is in place.

Sorting

Unit III. 20

Dr. Rabins Porwal

20

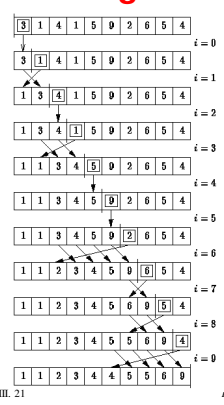


Internal Sorting

Insertion Sorting

Pros:
Relatively simple and easy to implement.

Cons:
Inefficient for large lists.



Sorting

Unit III. 21

Dr. Rabins Porwal

21



Internal Sorting

Algorithm InsertionSort

Input: An array $A[1, 2, \dots, n]$, where n is the number of elements.

Output: An array A with all elements in sorted order.

Steps:

1. For $i = 2$ to $n - 1$ do
2. $k = A[i]$
3. For $j = i - 1$ to 1 do
4. If $(k < A[j])$ then
5. $A[j+1] = A[j]$
6. EndIf
7. EndFor
8. $A[j] = k$
9. EndFor
10. Stop

Sorting

Unit III. 22

Dr. Rabins Porwal

22



Internal Sorting

Quicksort

An in-place, divide-and-conquer, massively recursive sort.

Algorithm is simple in theory, but very difficult to put into code.

Sorting

Unit III. 23

Dr. Rabins Porwal

23



Internal Sorting

Quicksort

Recursive algorithm consists of four steps:

1. If there is one or less element in the array to be sorted, return immediately.
2. Pick an element in the array to serve as a "pivot" point. (Usually the left-most element in the array is used.)

Sorting

Unit III. 24

Dr. Rabins Porwal

24



Internal Sorting

Quicksort

- Split the array into two parts - one with elements larger than the pivot and the other with elements smaller than the pivot.
- Recursively repeat the algorithm for both halves of the original array.

Unit III. 25

Dr. Rabins Porwal

25



Internal Sorting

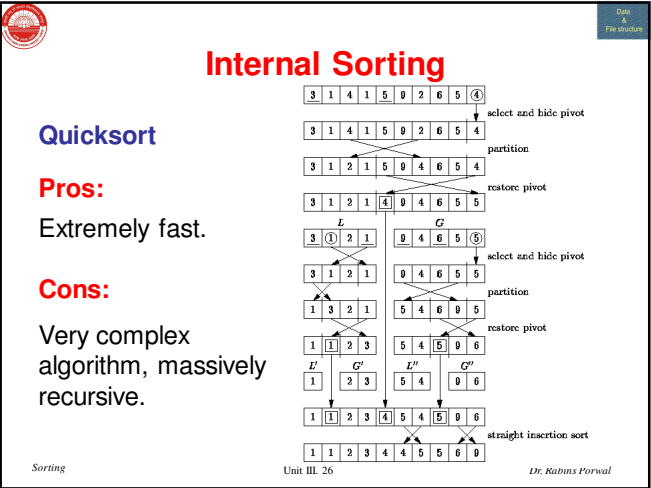
Quicksort

Pros:
Extremely fast.

Cons:
Very complex algorithm, massively recursive.

Unit III. 26

Dr. Rabins Porwal



26



Algorithm QuickSort

Input: A list of elements stored in an array A[low ... high], where low and high are two boundaries.

Output: Elements in A are stored in ascending order.

Steps:

- left = low, right = high
- If (left < right) then
- loc = Partition(A, left, right)
- QuickSort(A, left, loc-1)
- QuickSort(A, loc+1, right)
- EndIf
- Stop

Unit III. 27

Dr. Rabins Porwal

27



Algorithm Partition

Input: A list of elements stored in an array A. The leftmost and rightmost elements of the list under partition are located at left and right, respectively.

Output: Returns loc as the final position of the pivot element.

Steps:

- loc = left // The leftmost element is chosen as the pivot element
- While (left < right) do
- While (A[loc] ≤ A[right]) and (loc < right) do
- right = right - 1
- EndWhile
- If (A[loc] > A[right]) then
- Swap(A[loc], A[right])
- loc = right
- left = left + 1
- EndIf
- While (A[loc] ≥ A[left]) and (loc > left) do
- left = left + 1
- EndWhile
- If (A[loc] < A[left]) then
- Swap(A[loc], A[left])
- loc = left
- right = right - 1
- EndIf
- EndWhile
- Return (loc)
- Stop

Unit III. 28

Dr. Rabins Porwal

28



Internal Sorting

Heap Sort

- The slowest of the fast sorting algorithms, but does not require massive recursion or multiple arrays to work.
- This makes it the most attractive option for very large data sets of millions of items.

Unit III. 29

Dr. Rabins Porwal

29



Internal Sorting

Heap Sort

- Works as it's name suggests:
- Begins by building a heap out of the data set, and then removing the largest item and placing it at the end of the sorted array.

Unit III. 30

Dr. Rabins Porwal

30

Prepared by: Dr. Rabins Porwal

5



Internal Sorting

Heap Sort

- After removing the largest item, it reconstructs the heap and removes the largest remaining item and places it in the next open position from the end of the sorted array.

Sorting Unit III. 31 Dr. Rabins Porwal

31



Internal Sorting

Heap Sort

- This is repeated until there are no items left in the heap and the sorted array is full.

Sorting Unit III. 32 Dr. Rabins Porwal

32



Internal Sorting

Heap Sort

Pros:
In-place and non-recursive, making it a good choice for extremely large data sets.

Cons:
Slower than the merge and quick sorts.

Sorting Unit III. 33 Dr. Rabins Porwal

33



External Sorting

Generally carried out when serial files need to be converted to sequential files and there is insufficient space within the main memory of the system to hold all the data at once.

There are numerous algorithms which are used to perform sorts external to the computer's main memory.

Sorting Unit III. 34 Dr. Rabins Porwal

34



External Sorting

Among the more popular algorithms are:

- Tag Sorts
- Four Tape Sort
- Polyphase Sort
- External Radix Sort
- External Merge.

Sorting Unit III. 35 Dr. Rabins Porwal

35



External Sorting

Polyphase sort is the most efficient in terms of speed and utilisation of resources.
However, it is also most complicated.

In practice these sorting methods are already being supplemented by internal sorts.

Sorting Unit III. 36 Dr. Rabins Porwal

36



Unit III. 37

Dr. Rabins Porwal

External Sorting

Thus, a number of records from each tape would be read into main memory and sorted using an internal sort and then output to the tape rather than one record at a time as was the case initially.

37

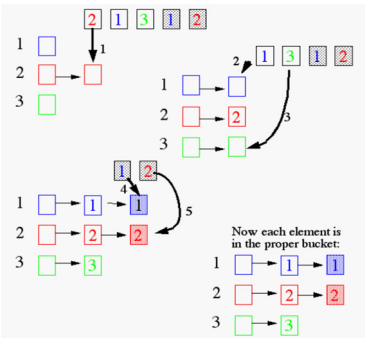


Unit III. 38

Dr. Rabins Porwal

Bucket Sort

Each element of the array is put in one of the N “buckets”



The diagram illustrates the bucket sort process. It starts with an array [2, 1, 3, 1, 2]. Three buckets are shown: bucket 1 (blue), bucket 2 (red), and bucket 3 (green). Elements are distributed into buckets: bucket 1 contains [1, 1], bucket 2 contains [2, 2], and bucket 3 contains [3]. The final state shows each element in its proper bucket, resulting in a sorted array [1, 1, 2, 2, 3].

38

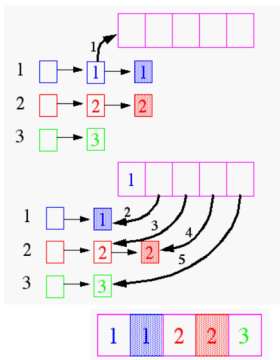


Unit III. 39

Dr. Rabins Porwal

Bucket Sort

Now, pull the elements from the buckets into the array



The diagram shows the final step of bucket sort. Elements are pulled from the buckets into the array in order. Bucket 1 (blue) contains [1, 1], bucket 2 (red) contains [2, 2], and bucket 3 (green) contains [3]. The final sorted array is [1, 1, 2, 2, 3].

At last, the sorted array (sorted in a stable way):

39